

Theory of Constraints + Black Belt in Thinking



Theory of Constraints and Black Belt in Thinking to focus on what really limits our results, think with clear cause and effect, and turn complexity into simple, practical action. We use TOC to find and improve the true constraint and BBiT to challenge assumptions, resolve conflicts, and design solutions that work end to end. By combining both, we stop firefighting, align everyone on the same goal, and create steady, measurable improvement in profit, flow, and quality of life at work.

Five Focusing Steps (Core of Constraint Management)

1 IDENTIFY



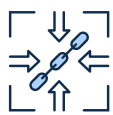
Find the system's constraint (physical, policy, paradigm, mindset, market).

2 EXPLOIT



Make the most of what you have (quick wins). Examples: reduce downtime, prioritize work for constraint, quality checks before it. Fix what's slowing the constraint without spending money yet.

3 SUBORDINATE



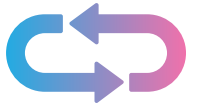
Align everything else to support the constraint. Non-constraints should never starve/block the constraint.

4 ELEVATE



If the constraint remains, take bigger actions, including investment. Invest only after exploit + subordinate are fully applied.

5 REPEAT



Once the constraint moves, start again. Never optimize everything. Always optimize the constraint. Avoid inertia.

TOC + BBiT step-by-step process



STEP 1: IDENTIFY THE UNDESIRABLE EFFECTS (UDE) / PROBLEMS

A clear statement of something that is not working. A symptom you can observe. UDEs are the starting point that help you see the problems objectively and later identify the root cause in the Current Reality Tree.

Use these 4 rules:



Describe only what you see

No explanations, no opinions.
Example: "Demos often start late."

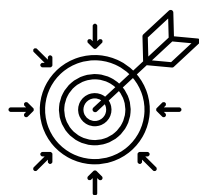
Describe the effect, not the cause

✗ "We have long sales cycles because qualification is weak."

✓ "50 percent of opportunities enter discovery without qualification."

Make it standalone and specific

Anyone should understand it.
Example: "Average deal cycle increased from 60 to 120 days."



Keep it neutral

No blame, no emotion. Example: "We rework proposals in 30 percent of cases."



How to collect them

?? What frustrates you? What keeps repeating? What slows customers down? What data shows a drop in performance? Where do handovers or communication regularly break?



Step 2: Build the Current Reality Tree (CRT)



The purpose of the CRT: find what to change. UDEs with if... then... causality. The CRT is complete when most UDEs link back to one or a few core drivers

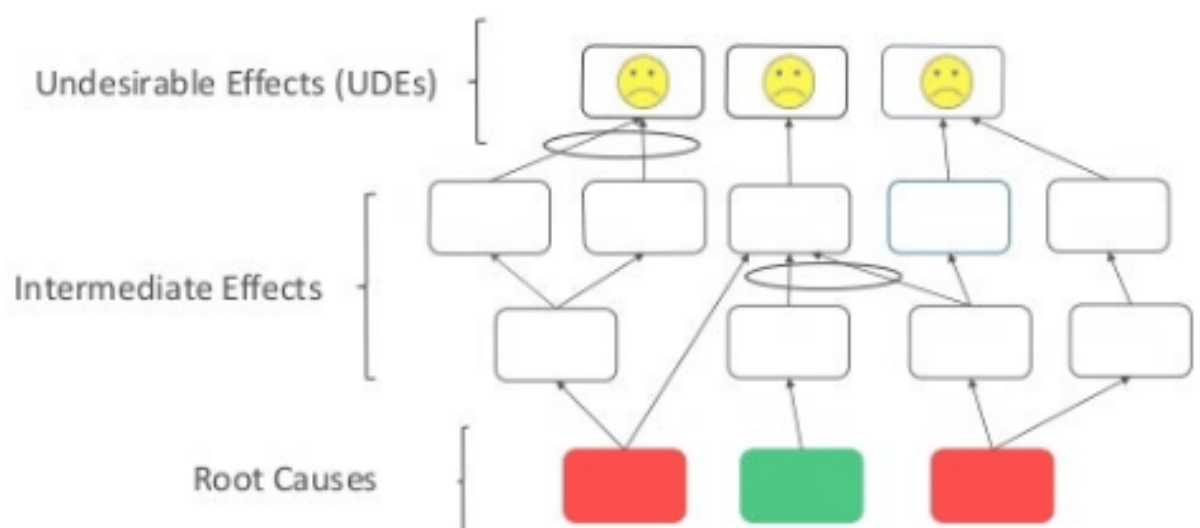


Causality test: Does the cause reasonably create the effect?

Assumption test: What hidden belief makes this link true?

Entity test: Are the statements themselves clear and valid?

This answers TOC Q1: What to change? ??



3

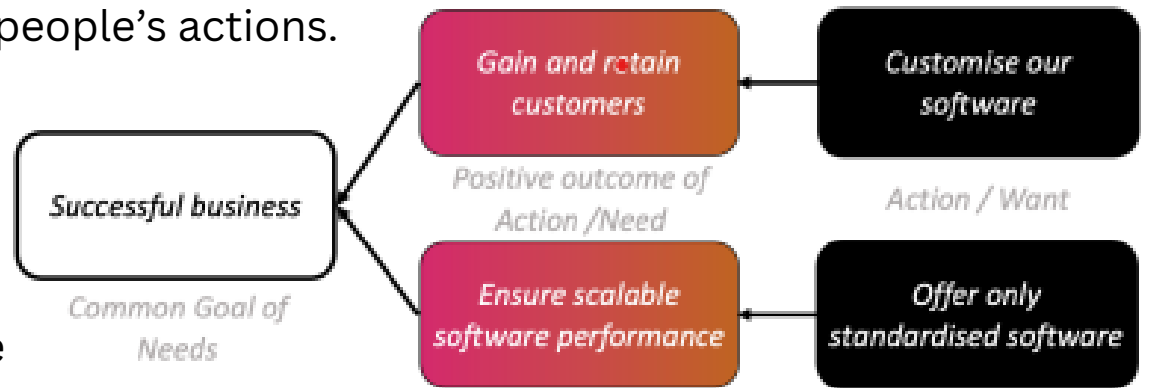
Step 3: Reveal the Conflict (Cloud)



Find the real conflict behind the symptoms and uncover the needs and assumptions that drive people's actions.

A Cloud appears when:

- Two actions compete
- Both actions make sense
- Both actions serve important needs
- You cannot choose both at the same time



4 List the assumptions

Identify the beliefs that link:
Action → Need
Need → Goal

3 Define the common goal

The goal that both needs support.

2 For each action, write the need it serves

Ask: "Why is this action important?" ??

1 Write the two competing actions (wants)

Word them neutrally, without hidden solutions.

5 Create injections: Break or dissolve the assumptions to make both needs achievable.



Define the conflict clearly. Ensure the two actions truly conflict (both are sensible but cannot co-exist) and each action is tied to a clear need. Write the actions and needs neutrally. No "hidden solutions" or opinion-based wording. Avoid naming a false conflict, using unclear needs, or phrasing actions as beliefs instead of concrete wants.

4

Step 4: Break the Assumptions (Druids)



You define the new state by breaking the assumptions that create the conflict. This answers TOC Q2: What to change to? ??

List assumptions that link actions → needs

Write each assumption as a clear, testable sentence.

List assumptions that link needs → mutual benefit

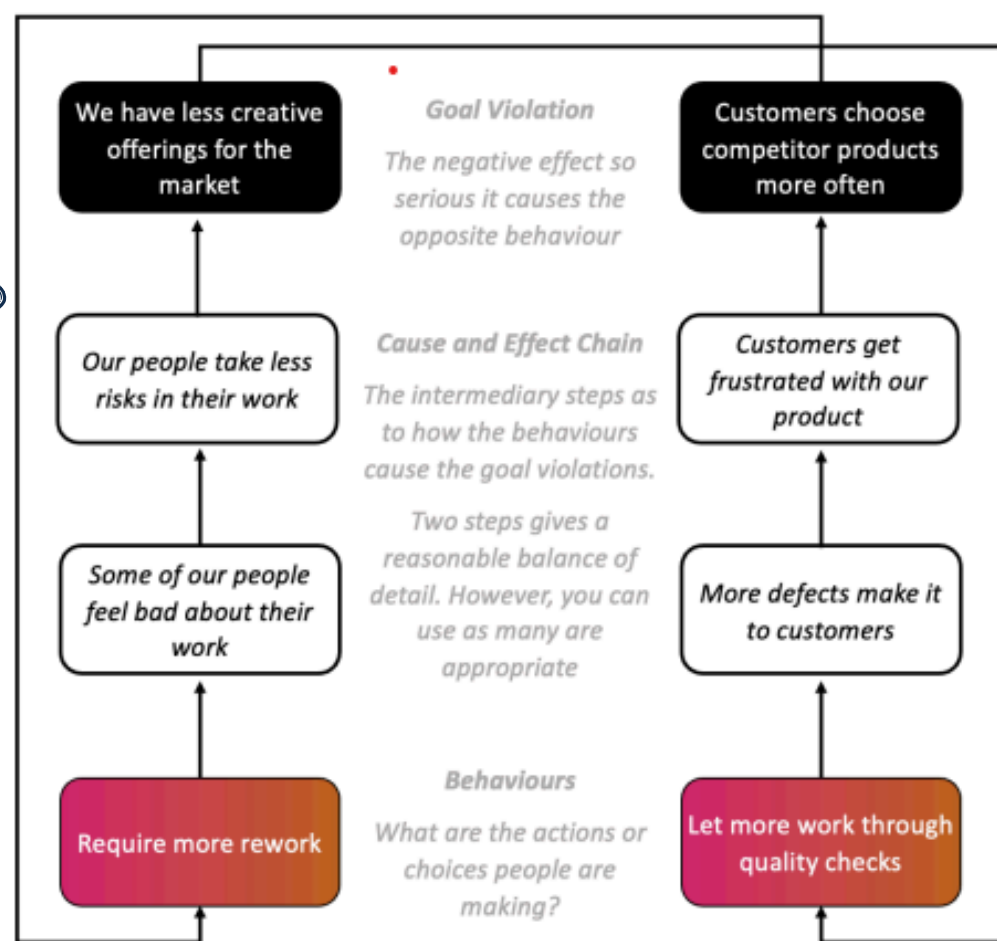
These assumptions explain why each need is required to protect the goal.

Challenge assumptions using logic tests

Is this always true? Does evidence support it? Is it a belief or a fact? Is it required for the link to hold? ??

Create multiple injections

Each injection should break or remove an assumption. Do not stop at the first one.



Druid: a method to map recurring conflict loops



An injection is a simple, practical change that breaks the assumptions holding a conflict or problem in place, so you can reach the desired outcome without compromise. Think of it as: A smart change that makes the problem disappear.

Select the best injection Choose the simplest and most practical injection that dissolves the conflict.

Check for win-win

- Both needs remain satisfied
- The conflict disappears
- No new negative effects are created

Use Druids for recurring or oscillating conflicts

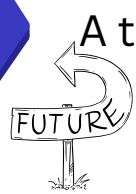
Druids expose negative loops that pull you back into the same problem. Break loops with injections targeted at violated needs.



Make each injection count. Every injection should target and break a specific underlying assumption that sustains the conflict. Don't settle for a one-sided, win-lose fix or a shallow tweak that leaves the core problem untouched, the solution must satisfy both needs without new negatives.

5

Step 5: Build the Future Reality Tree (FRT)



A tool that shows how your solution will work in the future and checks if it creates any new problems. The FRT answers TOC Q3: How to cause the change safely? ??
It ensures your solution works in reality, not just in theory.

4 Check for completeness

All injections should naturally lead to the final desired outcome or goal.

5 Identify Negative Branch

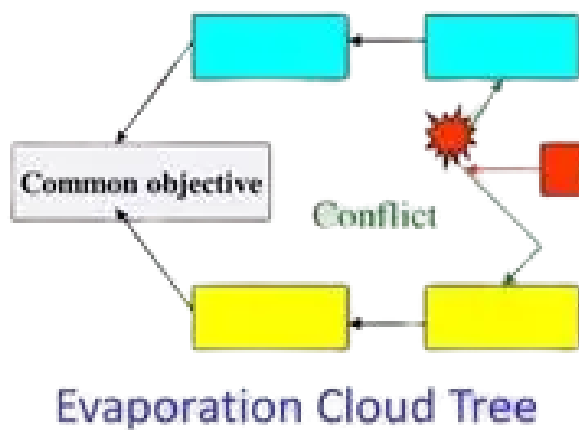
Reservations (NBRs) What could go wrong if we apply this injection? What negative side effect might appear? ??

7 Confirm the win-win state

The FRT must show that: all conflicts are dissolved, both needs are satisfied the system improves without new UDEs, the final goal is achieved

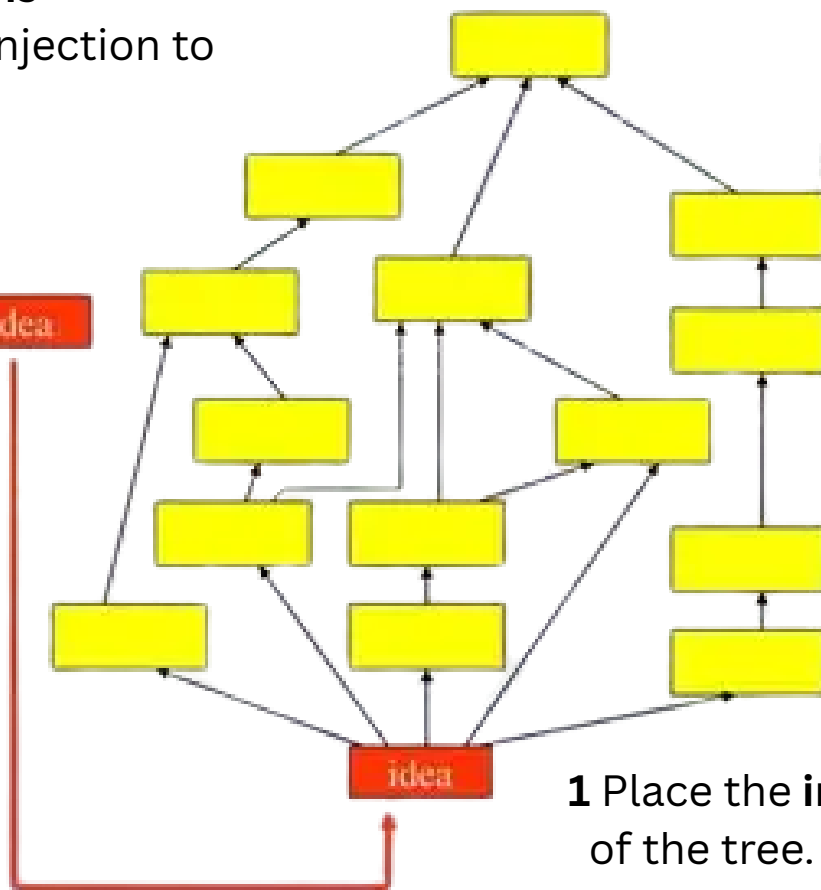
6 Add preventative injections

For each NBR, add a small targeted injection to neutralize the risk.



3 Validate each link using:

- Causality test: Does the cause truly create the effect?
- Assumption test: What must be true for this link to hold?
- Sufficiency test: Is the cause enough, or do we need additional injections?



2 Build upward using if... then... logic.

For each injection, ask: If we apply this injection, then what good effect happens? Link effects to other effects until you reach the high-level goal.

1 Place the injections at the bottom of the tree. These are the changes you want to introduce.



Build the Future Reality Tree with care. Validate each proposed cause-effect step using logic tests (check causality, assumptions, and sufficiency), and actively look for possible negative side effects ("negative branches") so you can address them in advance. Skipping these checks risks letting new problems slip by unnoticed.

6

Step 6: Plan via the Prerequisite Tree (PRT)

A PRT must start with one clear goal: the future state you want after the injections work. Write the goal first, before identifying obstacles

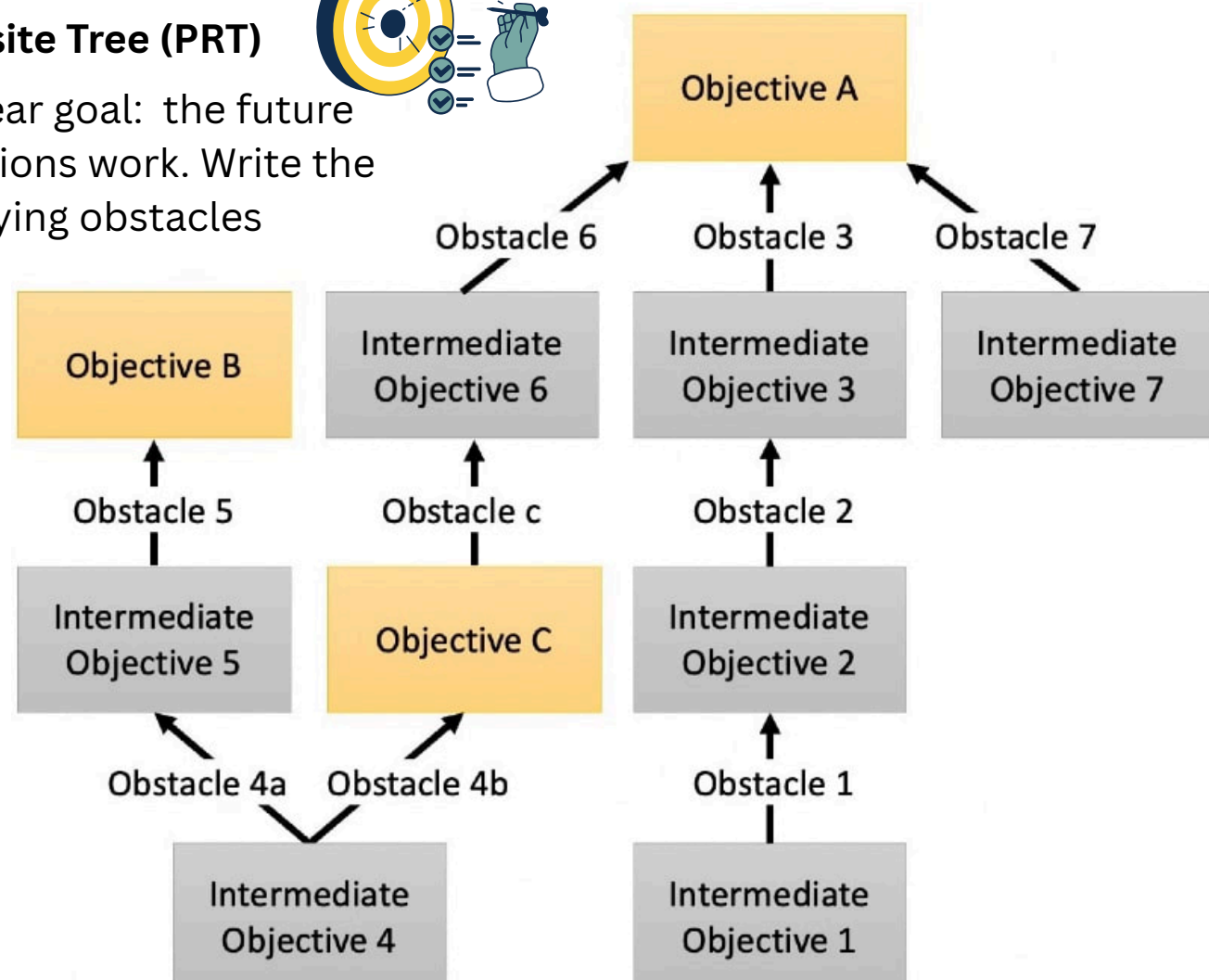


An obstacle is a statement of what prevents you from reaching the goal, not a missing action.

✗ "We haven't configured the system."

✓ "The system is not configured."

Intermediate Objectives (IOs) must be formulated as: "The obstacle is no longer present because..."



Validate the logic using: ??

Necessity test: Does this IO need to exist for the next IO to happen?

Assumption test: What assumptions make this dependency true? Are they valid?

Entity test: Are obstacles and IOs written as clear, standalone statements?

Critical Thinking Common Errors: Check for missing causes, circular logic, time-sequence confusion, etc.



When stuck, return to the CRT or Cloud to ensure you're addressing the real root cause or conflict



In the Prerequisite Tree, phrase obstacles and objectives properly. State each obstacle as a current condition that blocks the goal (not as a task you haven't done), and define each Intermediate Objective as the state where that obstacle is removed (the obstacle "no longer exists"). Avoid writing obstacles/IOs as action items or leaving their meaning ambiguous, each should clearly describe a condition to achieve.



Step 7: Execute via the Transition Tree (TT)

The TT turns the PRT into executable steps. It shows exactly what to do, why it works, and how each action moves the system forward. This answers TOC Q3: How to cause the change?

1 Current State (CS)

Describe the real situation at this moment. "Requirements are still unclear."

2. Action (A)

The concrete step you take next. Must be tied to an Intermediate Objective (IO). "Hold a structured clarification workshop with the customer."

3. Expected Effect (EE)

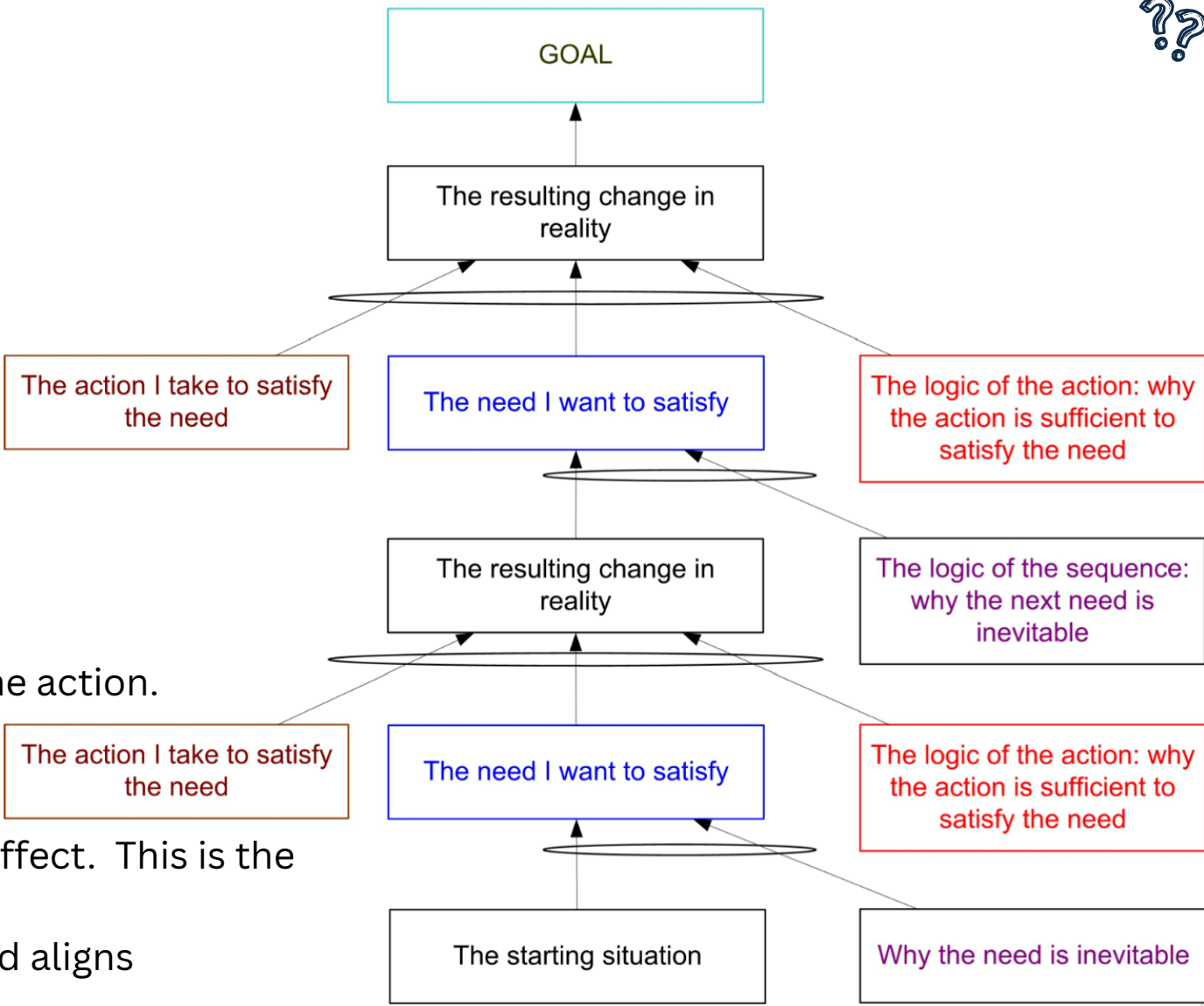
What you believe will happen after the action. "Requirements become clarified."

4. Logical Reason (R)

Explain why this action leads to the effect. This is the BBIT causal thinking requirement. "Clarification removes ambiguity and aligns expectations."

5. Actual Effect (AE)

Check whether the expected effect actually happened. If not, adjust the next action. "Requirements clarified and documented."



Use the new state as the starting point for the next CS → A → EE → R → AE chain. Continue until the final goal is reached.



Use the Transition Tree to plan realistic, step-by-step changes. Ensure each action is feasible from the current state and include a clear reason why that action will lead to the expected effect. Don't try to jump straight to the end state in one leap, and never omit the "why" – every step needs a logical explanation so you know how it moves you toward the goal.



- Q1 – What to change? (Steps 1 & 2: UDEs, CRT)
- Q2 – What to change to? (Steps 3 & 4: Cloud & Injections)
- Q3 – How to cause the change? (Steps 5, 6, 7: FRT, PRT, TT)